

Non-Blocking I/O

Non-Blocking I/O is a server design technique where a thread does **not wait** for I/O operations to complete.

Instead of blocking on a socket, the thread immediately gets control back and can continue doing other work.

This is a key building block of scalable systems.

1. Core Idea

Traditional blocking model:

```
None  
accept()  
  
read()  
  
wait until data arrives
```

Non-blocking model:

```
None  
accept()  
  
If no data → return immediately  
  
Thread continues running
```

2. How It Is Enabled

A socket file descriptor is configured with:

None

`O_NONBLOCK`

This tells the operating system:

None

`Do not pause thread if operation cannot complete now`

3. What Happens with accept()

If server calls:

None

`accept()`

and no pending client connection exists:

Instead of blocking, OS returns:

None

`EAGAIN`

`or`

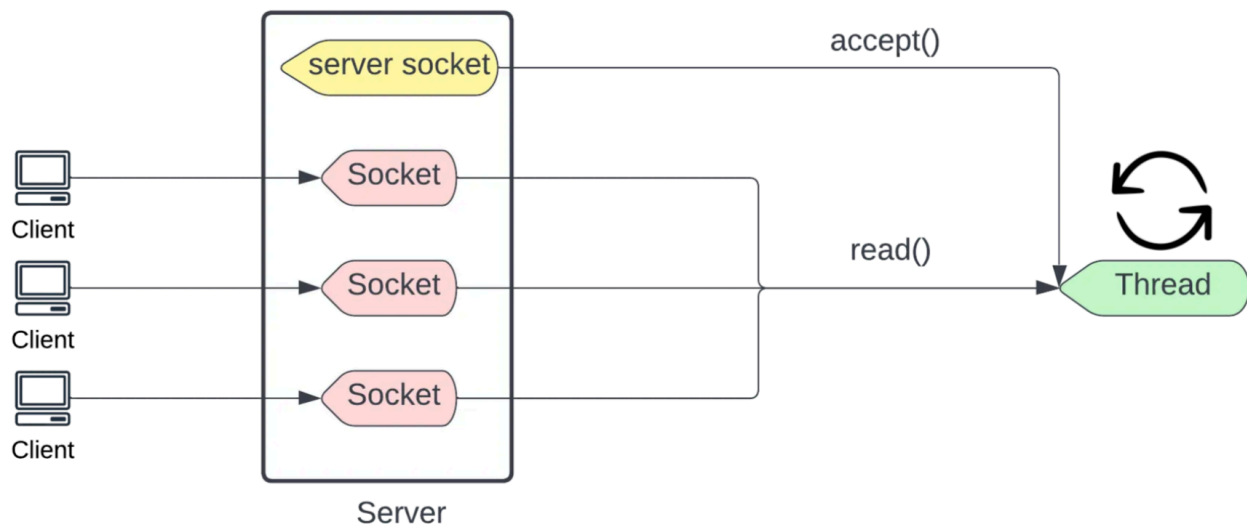
`EWOULDBLOCK`

Meaning:

None

No work available right now

Try again later



4. When New Connection Arrives

Once a client connects:

None

`accept()` returns new client socket descriptor (cfd)

Then server can manage that client connection separately.

5. Reading Client Data

Similarly for:

```
None  
read(cfd)
```

If data not ready:

```
None  
Returns EAGAIN / EWOULDBLOCK
```

Thread remains free.

6. Polling Approach

A naive non-blocking server repeatedly checks descriptors:

```
None  
while(true):  
    try accept()  
    try read(client1)  
    try read(client2)  
    try read(client3)
```

This is called:

None

Polling / Busy Waiting

7. Why Polling Is Bad

Even though thread is not blocked:

None

CPU keeps checking sockets repeatedly

This causes:

- Wasted CPU cycles
- High power usage
- Poor efficiency at scale

8. Non-Blocking but Still Synchronous

This model is:

None

Non-blocking = thread doesn't wait

Synchronous = thread manually keeps checking

So it improves over blocking I/O, but still not ideal.

9. Better Next Step: I/O Multiplexing

Instead of checking thousands of sockets manually, OS can notify ready sockets using:

- `select`
- `poll`
- `epoll` (Linux)
- `kqueue` (BSD/macOS)

Used by:

- Redis
- NGINX
- Node.js

10. Why It Matters in System Design

Non-blocking I/O enables:

- Fewer threads
- Better concurrency
- Lower memory usage
- Higher throughput

11. Comparison

Model	Thread Waits?	CPU Waste	Scalability
-------	---------------	-----------	-------------

Blocking I/O	Yes	Low while idle	Poor
Non-blocking + Polling	No	High	Medium
Non-blocking + epoll	No	Low	Excellent

Mental Model

None

```
Non-blocking I/O = Ask for work now, continue if none available
```

One-Line Summary

Non-blocking I/O allows a server thread to continue running when sockets are not ready, improving concurrency, but naive polling wastes CPU until combined with I/O multiplexing techniques like epoll.